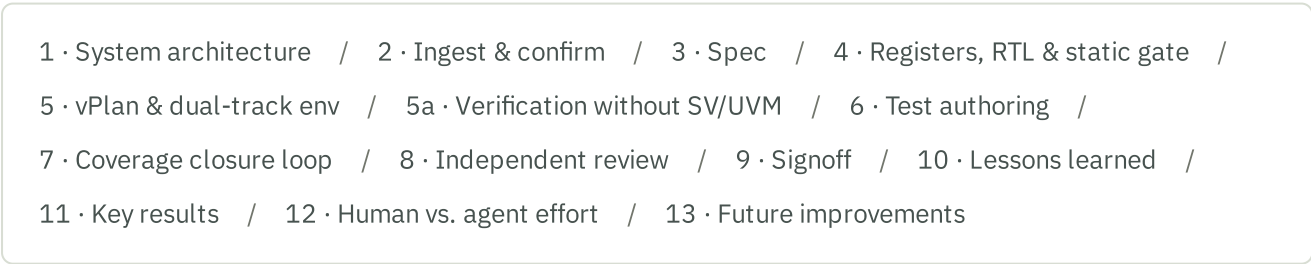
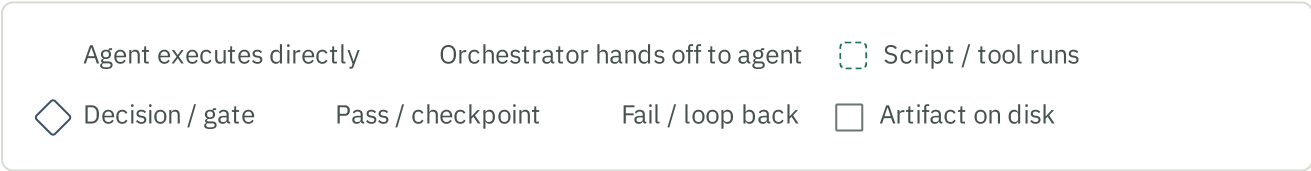
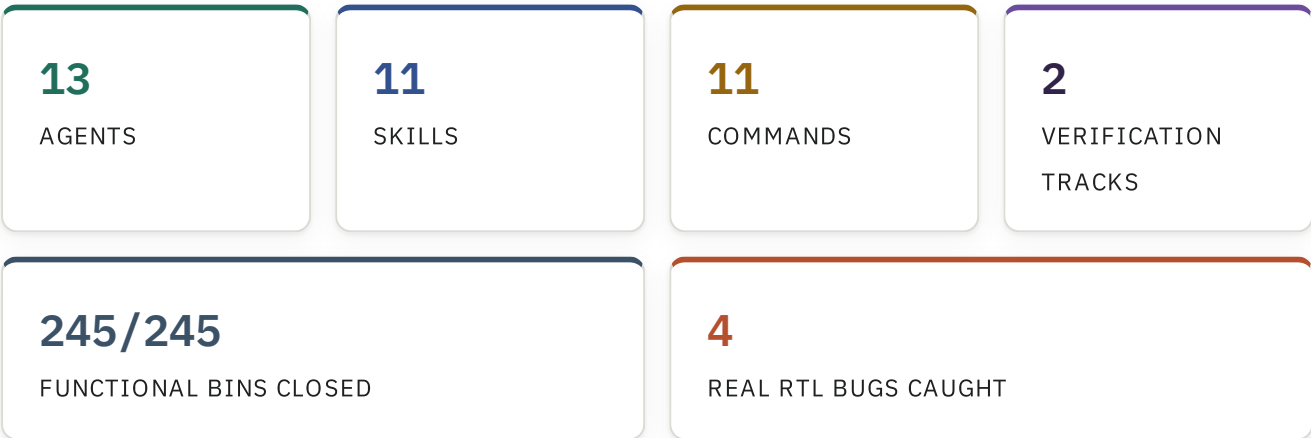


ARCHITECTURE & EXECUTION DATA FLOW

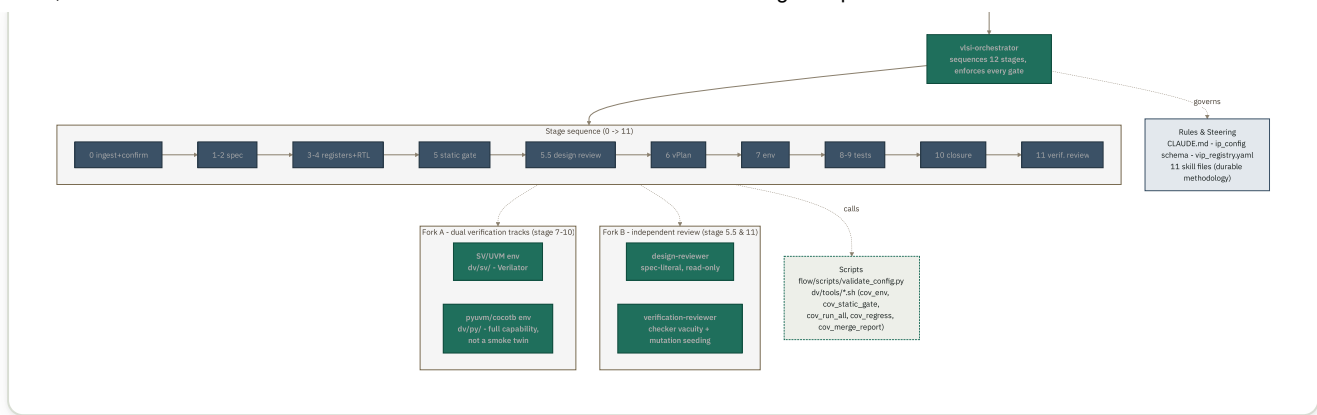
How the front-end VLSI agentic framework carries an IP from a requirement document to proven coverage closure — every stage, every gate, every rule that governs it — documented against its first real run: PMTPC-4, a 4-channel timer/PWM controller.



01 System Architecture

One orchestrator, thirteen specialists, two places the pipeline genuinely forks in parallel.

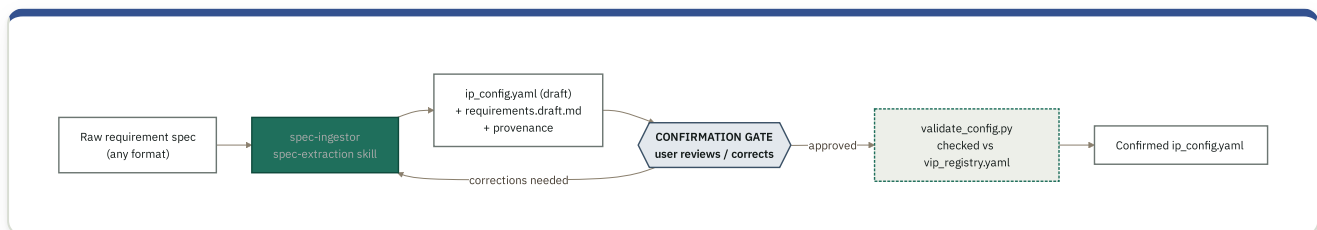




Unlike a fixed hub-and-spoke coverage tool, this framework spans the *whole* front-end lifecycle — the orchestrator's 12 stages are mostly sequential and gated, not fanned out. Genuine parallelism happens in exactly two places: the dual-track environment (SV/UVM and pyuvn/cocotb, generated from the same config so they can't diverge) and the two independent review agents, each running in fresh context with no stake in the changes it's checking.

02 Stage 0 — Ingest & Confirm

Agent: spec-ingestor · The front door. Nothing downstream runs until a human approves.



CLAUDE.md Generic-first: nothing hardcodes an IP's details — everything reads from `ip_config.yaml` .

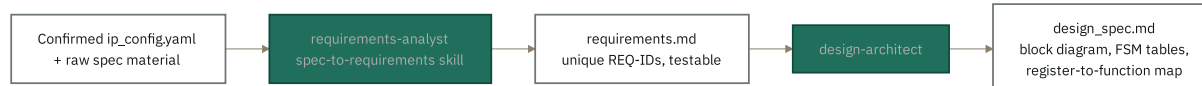
orchestrator.md "The ingestion confirmation gate is mandatory." Never run design/RTL/verification on an unconfirmed, auto-extracted config.

spec-ingestor.md Every extracted assumption is tagged in `provenance.needs_confirmation` — surfaced, never buried.

✓ Confirmed config validates against schema + VIP registry

03 Stage 1-2 — Requirements & Design Spec

Agents: requirements-analyst, design-architect · Builds the root of the traceability spine.



spec-to-requirements Every requirement gets a stable, atomic, testable ID — never reused or silently renumbered.

CLAUDE.md

Traceability chain begins here:

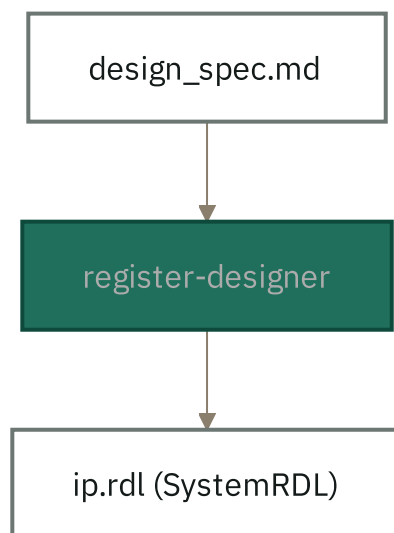
REQ → SPEC → REG → RTL → VPLAN → COV .

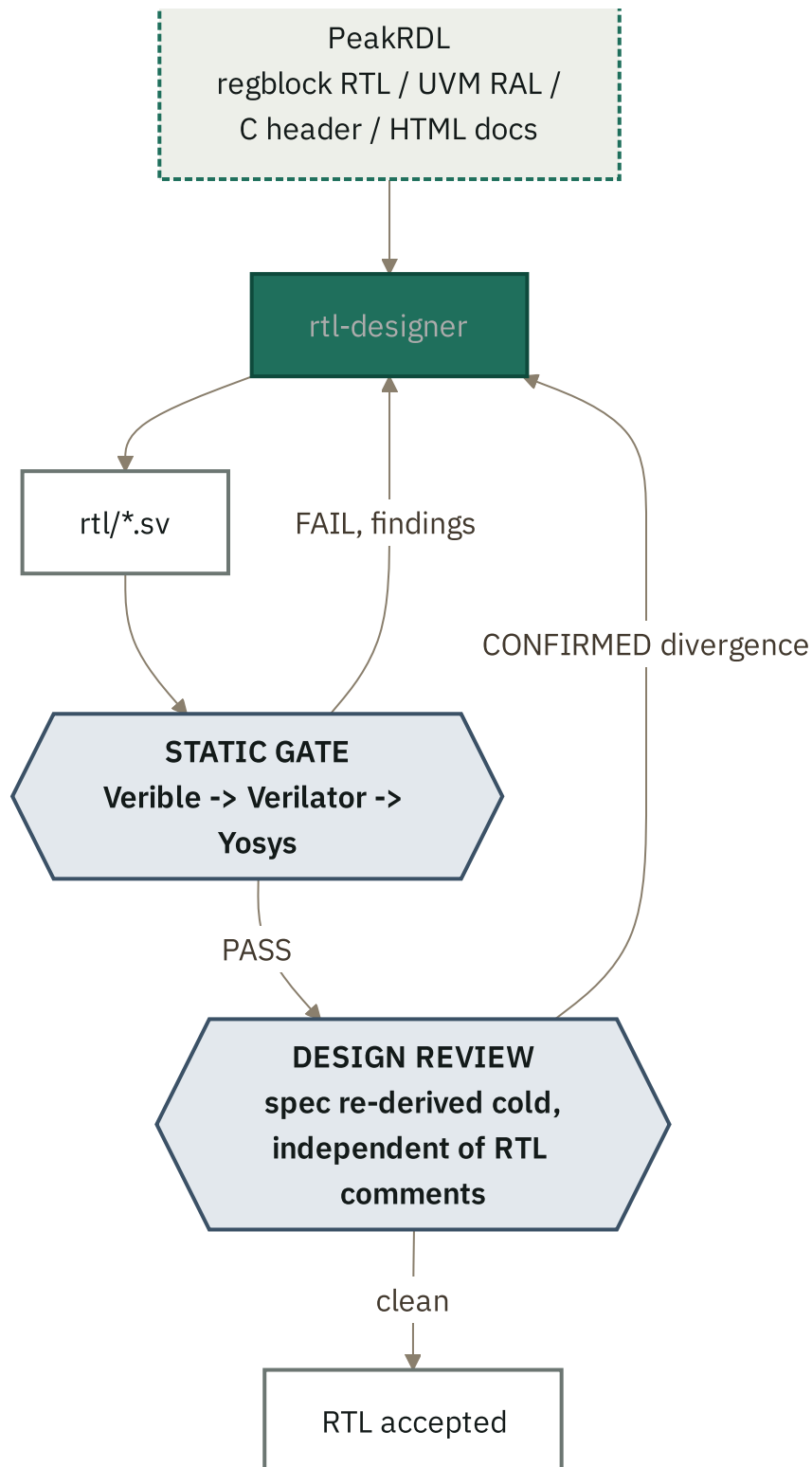
Real example — PMTPC-4: 22 requirements across 6 categories (core timer/PWM, interrupts, APB interface, reset/control, non-functional) — each traced all the way to a covergroup bin in the final closure report.

04

Stage 3-5.5 — Registers, RTL & the Static / Design Gates

Agents: register-designer, rtl-designer, lint-static-checker, design-reviewer · Two independent gates, not one.



**lint-static-checker.md**

peakrdl-regblock's

field_combo

struct trips Verilator's

MULTIDRIVEN

on every IP — waived once, generically, in a shared
filename-matched

flow/tools/verilator_waivers.vlt , never per-IP.

rtl-designer.md A bare

```
'0                on an unpacked
hwif_in           /
hwif_out          struct compiles under Yosys/sv2v but fails Verilator's C++ codegen — use
'{default:'0}     .
```

rtl-designer.md Never start a comment with the literal word "verilator" — it's parsed as a pragma, not prose.

design-review skill "Takes effect at the next X, not immediately" in a spec implies a shadow register latched at that event — check

every field described that way, not just the first one implemented.

Real example — PMTPC-4: four RTL bugs shipped past the static gate before being caught: a protocol phase decoded from a registered instead of combinational state (READY one cycle late on every transfer), a skipped FSM state on periodic reload, a missing hardware-clear on SOFT_RESET, and a mid-RUNNING COMPARE write applying live instead of shadowed. The design review also cross-referenced the Register Spec against the Design Spec's own FSM table and confirmed a fifth, initially-suspected gap (CH_EN=0 during LOAD/EXPIRED) was actually already correct — the Register Spec's field description said "regardless of current state," which the Design Spec's transition table alone didn't state.

✓ Verible + Verilator + Yosys clean, and design review finds no confirmed divergence

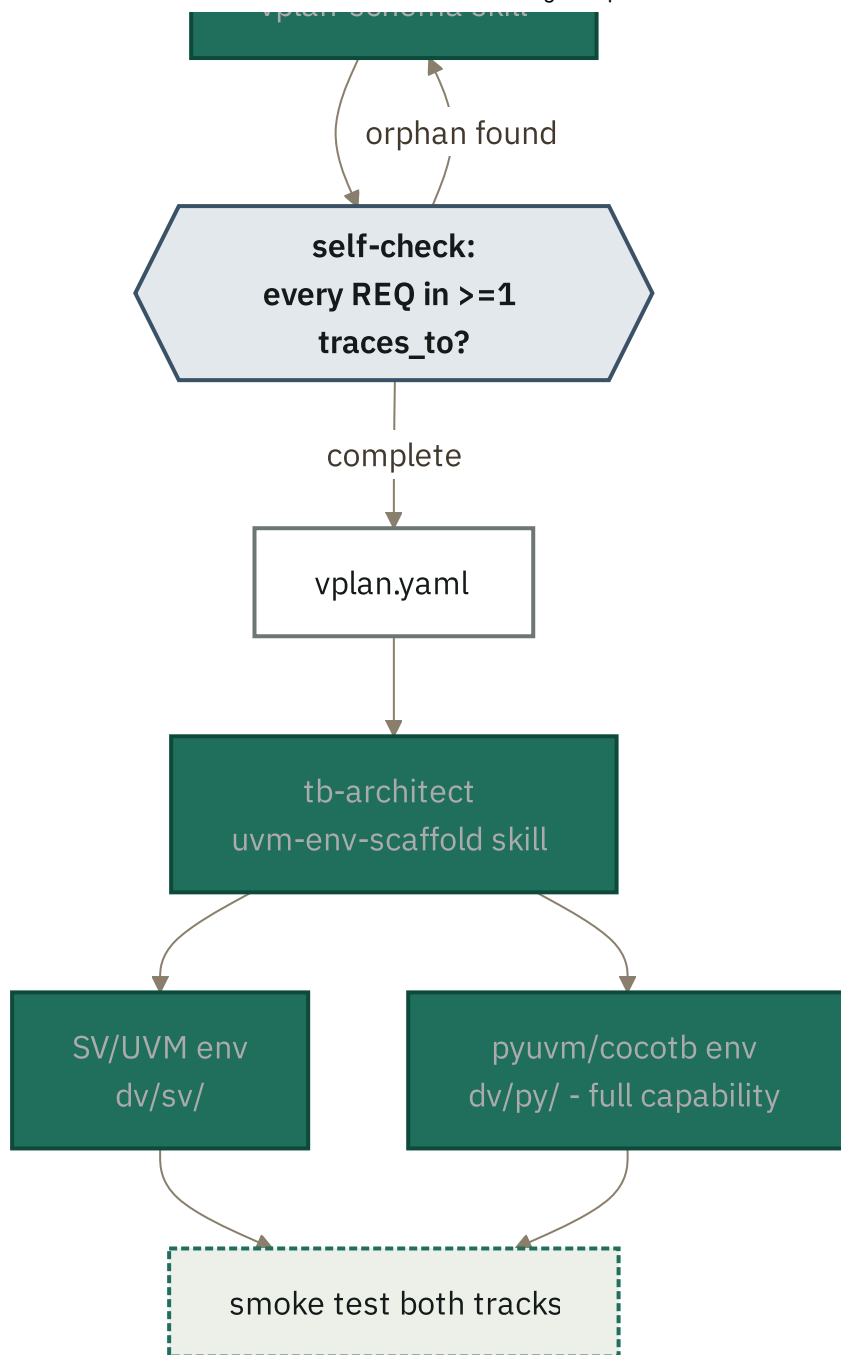
05

Stage 6-7 — Verification Plan & Dual-Track Environment

Agents: verification-planner, tb-architect · Both tracks generated at full capability — never one deep, one thin.

requirements.md +
design_spec.md + ip.rdl

verification-planner
vplan-schema skill



vplan-schema No orphan vPlan items, no requirement left untraced — checked here, before RTL/env effort is spent, not just flagged later at closure.

uvm-env-scaffold pyuvm/cocotb is generated at full, closure-grade capability — reframed from "smoke twin" after Verilator's UVM support turned out to be a hard wall, not an occasional gap.

tb-architect.md Confirm Verilator can elaborate bare `uvm_pkg.sv`, and that its version is ≥ 5.036 (cocotb's hard minimum), before investing in the SV/UVM track.

Real example — PMTPC-4: even Verilator built from source at a recent, UVM-focused tag crashed

(an internal compiler fault) elaborating Accellera UVM's own core library, before any IP-specific code ran. pyuvm/cocotb carried 100% of coverage closure alone; dv/sv/ is authored but carries no executed evidence — a documented gap, not a silent one.

05a

Verification Without SV/UVM

The question every reader asks first: if SV/UVM never executed, what actually did the checking?

The short answer: pyuvm is not "instead of UVM" — it *is* UVM. Same layered methodology (driver, sequencer, monitor, scoreboard, env, test, phases, sequences, factory), reimplemented in Python instead of SystemVerilog, running under cocotb instead of a SV simulator kernel. What's missing on PMTPC-4 isn't verification methodology — it's specifically the *SystemVerilog dialect* of UVM, blocked by a Verilator compiler bug in the Accellera SV class library (Stage 5a's neighbor, Stage 5, above).

Concept map: what replaced what

SV/UVM CONCEPT	WHAT ACTUALLY RAN INSTEAD	WHERE
Driver / Sequencer	cocotb coroutines drive psel/penable/paddr/pwdata directly onto DUT pins every clock edge via VPI	pmtpc4_apb.py
Monitor	cocotb reads DUT outputs (prdata, pready, pwm_out, irq, internal FSM state) every single clock edge	pmtpc4_cycle.py
Scoreboard / reference model	A cycle-accurate software model of the DUT, written independently from the spec — not from the RTL	pmtpc4_model.py
Functional covergroups	A Python Coverage class with named bins, sampled by the monitor only when a check just passed	pmtpc4_cov.py
Code coverage (line/branch/toggle/FSM)	Verilator's own native RTL instrumentation — agnostic to what language is driving the simulation	--coverage flag
SVA protocol assertions	Authored, not yet wired into the executed track — a real, currently-unused hybrid opportunity (see Lessons Learned)	dv/sv/sva/*.sv
Waveform review	Generated every run (--trace); spot-checked by hand on request, not as the	dump.vcd

primary gate

Step by step: how a single cycle actually gets checked

- 1 `cocotb` drives the APB bus signals for one transaction (e.g. a write to `CH0_COMPARE`).

- 2 On every `RisingEdge(pclk)`, the monitor reads the *actual* DUT signals: `prdata`, `pready`, `pslverr`, `pwm_out`, `irq`, `INT_STATUS`, and — via Verilator's whitebox access — every channel's internal FSM `state` and `count`.

- 3 In the same cycle, it feeds that cycle's bus inputs into the reference model's `step()` function, which independently computes what those signals *should* be, from the spec's rules — not by reading the RTL.

- 4 It compares actual vs. expected, signal by signal. Any mismatch increments an error counter immediately.

- 5 **Only if that cycle's checks passed** does it sample a coverage bin — coverage is a side effect of a check that already succeeded, never sampled independently by test code.

- 6 This repeats for every cycle of all 26 tests, across 25 randomized seeds. One single test run logged **33,160 individual cycle-checks, 0 errors** — exhaustive comparison at a scale no manual waveform review reaches.

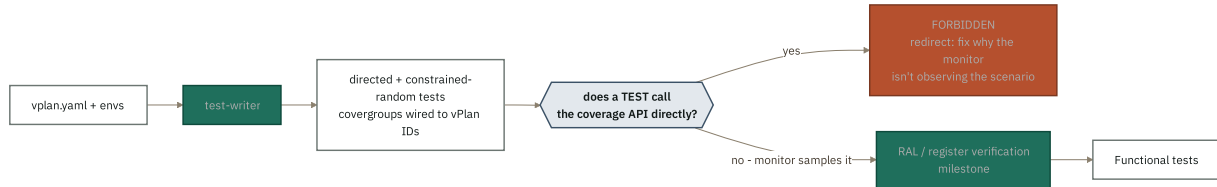
- 7 `coverage-closure` merges the functional bins with Verilator's independent code coverage, triages every unhit bin (fix it, waive it with a structural reason, or file it as a bug), and loops until both hit 100%.

- 8 `verification-reviewer` then **deliberately broke the RTL twice** (a flipped comparison operator, a removed error-check gate) and reran — confirming step 4's checks actually fail when the DUT is wrong, not just when it's right. That's the proof the scoreboard isn't decorative.

Real example — PMTPC-4: reading a generated dump `.vcd` by hand for `pwm_duty_test` (`COMPARE=0`, `PERIOD=8`) showed channel 0's PWM going high the instant count loads to 8, staying high through every decrement, and dropping low on the exact cycle count reaches 0 — a cycle-exact match to Design Spec §7.4's "continuously high except at `COUNT=0`" rule, confirmed by eye, not just by the automated check.

06 Stage 8-9 — Test Authoring

Agent: *test-writer* · The single most important rule in the whole pipeline lives here.



test-writer.md Coverage may only be sampled by the monitor/scoreboard, from a signal it just checked — never by test/sequence code calling

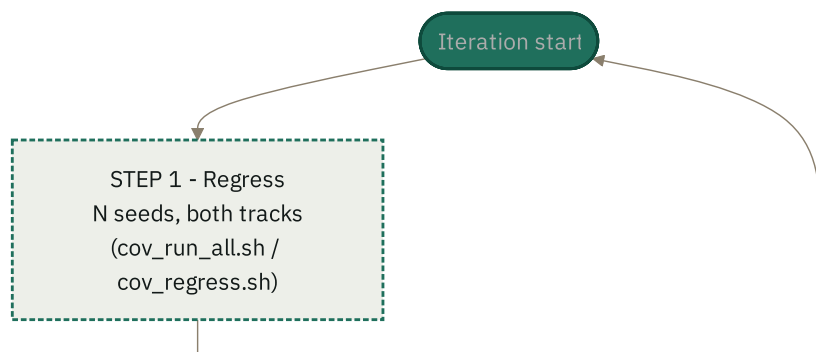
`.sample()` directly. A test's job is to
create a scenario, never to declare it covered.

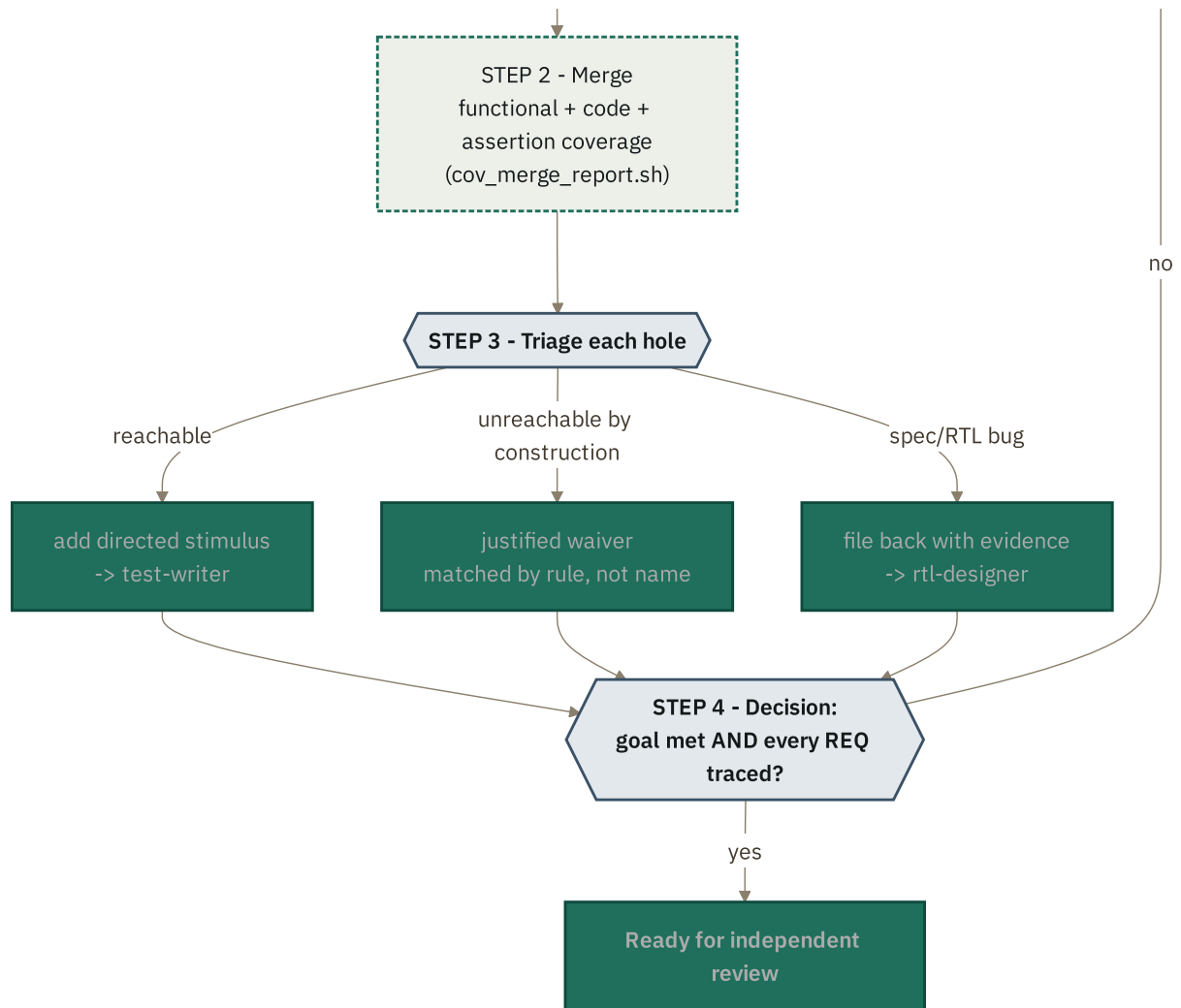
test-writer.md Every test maps to ≥ 1 vPlan ID; self-checking against the scoreboard/RAL, never eyeballing waveforms.

Real example — PMTPC-4: the first-generation environment reached "100%" functional coverage while checking almost nothing — tests called the coverage API directly to mark bins covered, disconnected from any signal actually observed. It was rebuilt around a cycle-accurate reference model (`pmtpc4_cycle.py`) that checks the DUT every clock and samples coverage only from signals it just checked. This one discovery is why `test-writer.md` now states the rule explicitly instead of leaving it implied.

07 Stage 10 — Coverage Closure Loop

Agent: *coverage-closure* · Regress, merge, triage, iterate — until the number is real.





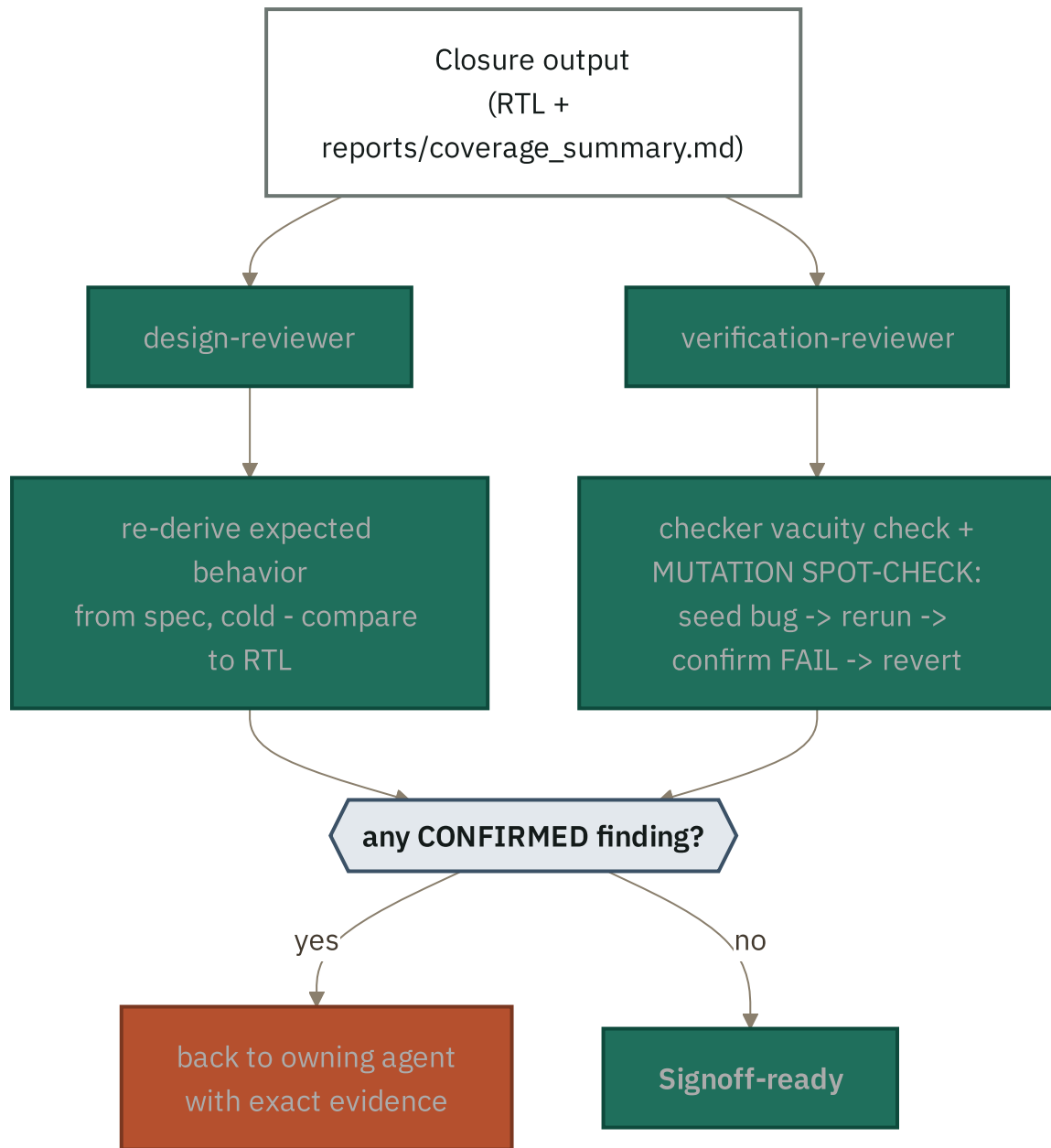
coverage-triage Classify every hole: reachable → stimulus, unreachable → justified waiver, bug → file back with evidence. Never silently drop one.

coverage-closure.md "100% functional coverage with failing/absent checks is not closure" — pass/fail and coverage must both be green, and the scoreboard must actually be checking.

verification-review skill Waivers matched by rule/pattern, never a hand-maintained allow-list of point names — a pattern keeps catching new holes; a name list silently stops.

Real example — PMTPC-4: 25 seeds × 26 tests, 0 failures. 245/245 functional bins (3 waived, unreachable by construction), 1927/1927 code-coverage points net of waivers (180 waived across 7 structural, rule-matched families). Four real RTL bugs were found and fixed *during* this loop, not just coverage holes.

Agents: design-reviewer, verification-reviewer · Fresh context, no stake in the changes just made.



design-review skill Rebuild the contract from the spec

before

reading the RTL — an inline comment's justification is a claim, never proof.

verification-review skill Mutation/bug-seeding is the standard way to prove a checker works: seed an obviously-wrong change, confirm something fails, always revert immediately — never leave a seeded bug in the tree.

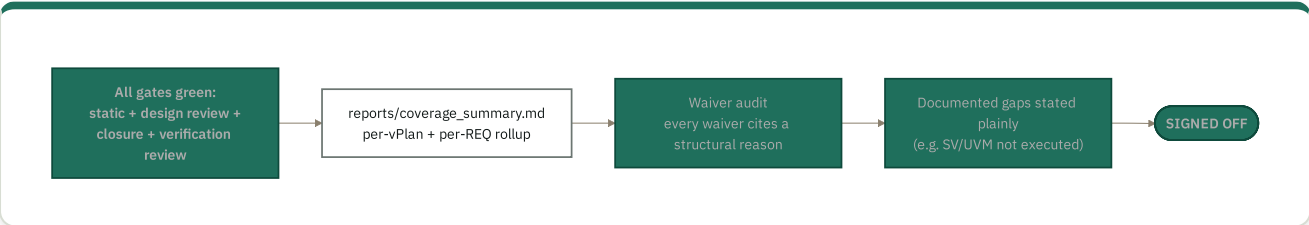
both skills Report, don't fix. Reviewing and fixing in the same pass re-introduces the

confirmation bias the review exists to avoid.

Real example — PMTPC-4: the verification-reviewer seeded two deliberate bugs — a comparison operator flip in the PWM channel core, and a removed write-gate in the APB error decode — in two different modules. Both were caught immediately by the existing checks (one via 6 reference-model errors, the other at three independent layers simultaneously). Both were reverted and confirmed clean via `git diff` before the full 26-test suite was re-run green.

09 Signoff

Every requirement closed, every waiver justified, every gap documented — not hidden.



245/245 functional bins, 100%

1927/1927 code points net of waivers, 100%

22/22 requirements closed

25/25 regression seeds green

10 Lessons Learned

Nine real discoveries from PMTPC-4's first run through the pipeline, and the permanent change each one produced.

#	DISCOVERY	IMPACT	CHANGE MADE
1	Static analysis missed a comparison operator flip in the PWM channel core.	6 reference-model errors caught the bug.	Reverted the flip and confirmed clean via git diff.
2	Verification reviewer removed a write-gate in the APB error decode.	Caught at three independent layers simultaneously.	Reverted the removal and confirmed clean via git diff.

1	nested quoting through <code>ws1 bash -lc</code> silently corrupts <code>\$(...)</code> substitutions	Commands appeared to run but used empty/wrong values, with no error	Convention: write multi-step commands to a script file, never inline
2	apt's Verilator 5.032 accepted as "installed" by a bare <code>grep "5\."</code>	Skipped the from-source build cocotb/UVM actually needs	<code>bootstrap-ws12.sh</code> now does a real ≥ 5.036 version comparison
3	<code>hwif_in = '0;</code> on an unpacked struct compiles under Yosys/sv2v, fails Verilator's C++ codegen	Static gate passed; simulation build failed	Use <code>'{default: '0};</code> documented in <code>rtl-designer.md</code>
4	A comment starting with the word "verilator" is parsed as a pragma	<code>BADVLT</code> PRAGMA compile error from plain prose	Documented as an explicit style rule
5	First-generation coverage model sampled bins directly from test code	"100%" coverage that checked almost nothing	Rebuilt around a cycle-accurate reference model; anti-pattern rule added to <code>test-writer.md</code>
6	<code>peakrdl-regblock's field_combo</code> struct trips Verilator <code>MULTIDRIVEN</code> on every IP	Would be rediscovered per-IP	Shared, filename-matched <code>flow/tools/verilator_waivers.vlt</code>
7	Verilator (even v5.038 from source) crashes elaborating UVM's own core library	SV/UVM unusable as the primary track	<code>pyuvn/cocotb</code> re-scoped to full-capability; SV/UVM reframed as best-effort
8	Mid-RUNNING COMPARE write applied live instead of shadowed (Design Spec §7.4)	Passed lint, elaboration, and first-pass sim	Caught only by literal spec re-read; shadow register added, model fixed to match

06/09/2026, 20:16		VLSI Agent Pipeline	
9	No independent review stage existed	Coverage-closure graded its own homework	design-reviewer / verification-reviewer added, validated day-one via 2 live mutation spot-checks

11 **Key Results – PMTPC-4**

4-channel timer/PWM controller, APB3, 5 binding errata. First IP carried end-to-end.

METRIC	VALUE
Requirements	22
vPlan items	25
Functional coverage bins	245 / 245 (100%)
Functional bins waived (unreachable)	3
Code coverage, net of waivers	1927 / 1927 (100%)
Code coverage, raw	1927 / 2107 (91.46%)
Code-coverage points waived	180, across 7 rule-matched families
Regression	25 seeds × 26 tests, 0 failures
RTL bugs found & fixed	4
Mutation spot-checks run / caught	2 / 2
SV/UVM track	blocked – Verilator ICE, documented gap
pyuvvm/cocotb track	sole executed proof – full closure

12 **Human vs. Agent-Assisted Effort**

Illustrative from this one real run — not a large-sample study, but every line below happened.

STAGE	WHAT THE AGENT DID ALONE	WHAT NEEDED THE HUMAN
Environment	Diagnosed a broken bootstrap script, Fixed the <code>iverilog</code> version flag bug	Ran the <code>sudo apt-get install</code> lines themselves (no interactive

Setup	fixed the iverilog version-tag bug, cloned/built Verilator from source, fetched UVM sources	times themselves (no interactive password channel available to the agent)
SV/UVM vs. pyuvm strategy	Diagnosed the exact Verilator ICE, tried two escalation paths (fetch UVM, build from source at a newer tag)	Decided when to stop chasing SV/UVM and commit to pyuvm as the proof track — a scope call, not a technical one
Coverage closure	Ran the full 25-seed regression, triaged every hole, rebuilt the coverage model, fixed 3 RTL bugs autonomously	Reviewing and accepting those 3 fixes as correct design changes
Spec ambiguity (COMPARE shadow)	Re-derived the correct behavior from the spec, implemented the fix, added a targeted test, re-proved closure	Flagged the ambiguity for the user's attention before it was resolved
Independent review	Built the review methodology from real signoff practice (OpenTitan checklist, mutation-testing literature), ran 2 live bug-seeding spot-checks end-to-end	Chose to create two review agents (design + verification) rather than one, on the user's explicit direction

12a

What Only Humans / Only Agents Can Do

ONLY HUMANS	
Grant sudo / interactive credentials	No interactive password channel reaches the